

# Configure MongoDB

## Overview

MongoDB is used as the primary database for the DarshanEase application. It stores user accounts, temple details, darshan slots, bookings, and administrator information. The application connects to MongoDB using **Mongoose**, an Object Data Modeling (ODM) library for Node.js.

## Prerequisites

Before configuring MongoDB, ensure the following are installed:

- Node.js (v18 or later)
- MongoDB Community Server **or** MongoDB Atlas
- npm (Node Package Manager)

Install Mongoose in the backend project:

```
npm install mongoose
```

## Option 1: Configure MongoDB Atlas (Recommended)

### Step 1: Create a MongoDB Atlas Account

- Visit the MongoDB Atlas website.
- Sign up or log in.
- Create a new project.

### Step 2: Create a Cluster

- Click **Build a Database**.
- Select the **Free Shared Cluster (M0)**.
- Choose a cloud provider and region.
- Create the cluster.

### Step 3: Create a Database User

- Go to **Database Access**.

- Create a username and password.
- Assign **Read and Write** permissions.

#### Step 4: Configure Network Access

- Open **Network Access**.
- Add your IP address or allow access from all IPs (0.0.0.0/0) for development.

#### Step 5: Get the Connection String

- Click **Connect** → **Drivers**.
- Copy the MongoDB connection URI.

Example:

```
mongodb+srv://username:password@cluster0.xxxxx.mongodb.net/darshanease?
retryWrites=true&w=majority
```

## Option 2: Configure Local MongoDB

Start the MongoDB service on your computer.

Create a database named:

darshanease

Local connection string:

```
mongodb://localhost:27017/darshanease
```

## Create the Environment File

Inside the **backend** folder, create a file named **.env**.

```
PORT=5000
```

```
MONGODB_URI=mongodb://localhost:27017/darshanease
```

```
JWT_SECRET=your_secret_key
```

```
NODE_ENV=development
```

For MongoDB Atlas:

```
PORT=5000
```

```
MONGODB_URI=mongodb+srv://username:password@cluster0.xxxxx.mongodb.net/
darshanease
```

```
JWT_SECRET=your_secret_key
```

```
NODE_ENV=production
```

# Create the Database Configuration File

Create **config/db.js**.

```
const mongoose = require("mongoose");

const connectDB = async () => {
  try {
    await mongoose.connect(process.env.MONGODB_URI);

    console.log("MongoDB Connected Successfully");
  } catch (error) {
    console.error("Database Connection Failed");
    console.error(error.message);
    process.exit(1);
  }
};

module.exports = connectDB;
```

## Configure server.js

Import and initialize the database connection.

```
require("dotenv").config();

const express = require("express");
const cors = require("cors");
const connectDB = require("./config/db");

const app = express();

// Connect to MongoDB
connectDB();

// Middleware
app.use(cors());
app.use(express.json());

app.get("/", (req, res) => {
  res.send("DarshanEase API Running...");
});

const PORT = process.env.PORT || 5000;
```

```
app.listen(PORT, () => {  
  console.log(`Server running on port ${PORT}`);  
});
```

## Test the Connection

Start the backend server.

```
npm run dev
```

If the configuration is successful, the terminal displays:

```
MongoDB Connected Successfully  
Server running on port 5000
```

## Verify the Database

After running the application:

1. Register a new user.
2. Log in to MongoDB Atlas or use MongoDB Compass.
3. Open the **darshanease** database.
4. Verify that collections such as **users**, **temples**, **slots**, and **bookings** are created automatically when data is inserted.

## MongoDB Collections

```
darshanease  
├── users  
├── temples  
├── slots  
├── bookings  
└── admins
```

## Advantages of MongoDB

- NoSQL document-based database.
- Flexible schema for application growth.
- High performance and scalability.

- Easy integration with Node.js using Mongoose.
- Supports cloud deployment through MongoDB Atlas.
- Well-suited for MERN stack applications.

## Conclusion

MongoDB provides a reliable and scalable data storage solution for the DarshanEase application. By configuring the database with Mongoose and managing connection details through environment variables, the backend can securely store and retrieve information related to users, temples, darshan slots, and bookings while maintaining good development practices.